



Programming Java

The Abstract Window Toolkit

Incheon Paik

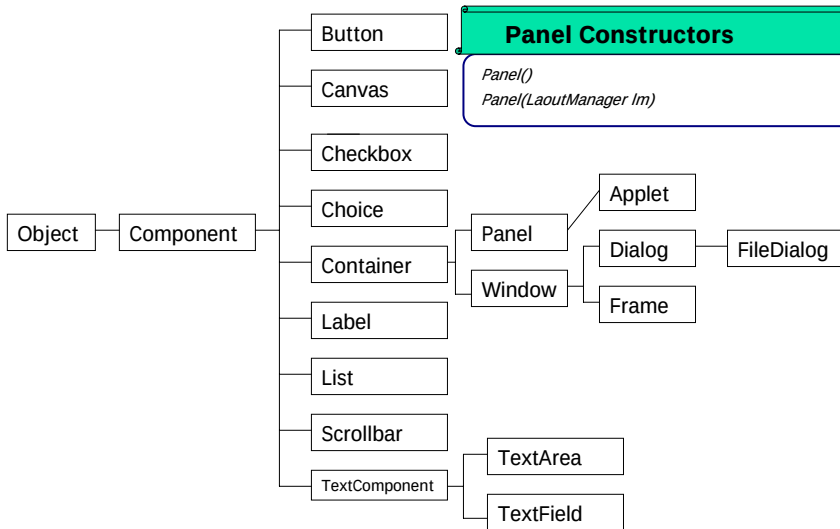


Contents

- Overview
- Labels
- Buttons
- Canvases
- Check Boxes
- Check Box Groups
- Choices
- Text Fields and Text Areas
- Lists
- Scroll Bars
- Layout Managers
- Border Layout
- Grid Layout and Insets
- Panels
- Windows and Frames
- Menus and Menu Bars
- Dialogs and File Dialogs



Overview



Panel Constructors

Panel()
Panel(LayoutManager lm)



Labels

Label Constructors

Label()
Label(String str)
Label(String str, int align)

Methods

int getAlignment()
void setAlignment(int align)
String getText()
void setText(String str)

```
public class Labels extends Applet {
```

```
    public void init() {
        String s = "This is a very long label";
        Label l1 = new Label(s, Label.LEFT);
        add(l1);
        Label l2 = new Label(s, Label.CENTER);
        add(l2);
        Label l3 = new Label(s, Label.RIGHT);
        add(l3);
    }
}
```

This is a very long label
This is a very long label
This is a very long label



Buttons

Button Constructors

```
Button()
Button(String str)
```

Methods

```
void addActionListener(ActionListener al)
void removeActionListener(ActionListener al)
String getLabel()
void setLabel(String str)
String getActionCommand()
void setActionCommand(String str)
ActionEvent(Object src, int type, String cmd)
ActionEvent(Object src, int type, String cmd, int modifiers)
String getActionCommand()
int getModifiers()
void actionPerformed(ActionEvent ae)
```

1. Create a Button object
2. Register the applet : addActionListener
3. Invoke the add() method
4. Implement the ActionListener

Java

5

AWT

Apple Banana Orange Banana

```
import java.applet.*;
import java.awt.*;
import java.awt.event.*;
/*
<applet code="ButtonEvents" width=400 height=60>
</applet>
*/
```

```
public class ButtonEvents extends Applet
implements ActionListener {
    Label label;
```

```
    public void init() {
        Button b1 = new Button("Apple");
        b1.addActionListener(this);
        add(b1);
        Button b2 = new Button("Banana");
        b2.addActionListener(this);
        add(b2);
        Button b3 = new Button("Orange");
        b3.addActionListener(this);
        add(b3);
        label = new Label("      ");
        add(label);
    }
```

```
    public void actionPerformed(ActionEvent ae) {
        label.setText(ae.getActionCommand());
    }
}
```



Canvases

Canvas Constructors

```
Canvas()
Can override the paint() method
```

```
import java.applet.*;
import java.awt.*;
/*
<applet code="CanvasDemo" width=501 height=200>
</applet>
*/

class MyCanvas extends Canvas {

    public void paint(Graphics g) {

        //Draw axes
        g.setColor(Color.lightGray);
        Dimension d = getSize();
        g.drawRect(0, 0, d.width - 1, d.height - 1);
        g.drawLine(0, d.height / 2, d.width - 1, d.height / 2 - 1);
        g.drawLine(d.width / 2, 0, d.width / 2 - 1, d.height - 1);
    }
```

//Draw waveform

```
g.setColor(Color.blue);
double dx = 4 * Math.PI / d.width;
double x = -2 * Math.PI;
int h = d.height;
for (int i = 0; i < d.width - 1; i++) {
    int y1 = (int)((h - h * Math.sin(x)) / 2);
    int y2 = (int)((h - h * Math.sin(x + dx)) / 2);
    g.drawLine(i, y1, i + 1, y2);
    x += dx;
}
}
```

```
public class CanvasDemo extends Applet {

    public void init() {
        MyCanvas myCanvas = new MyCanvas();
        myCanvas.setSize(401, 150);
        add(myCanvas);
    }
}
```



Java

6

AWT



Check Boxes

Checkbox Constructors

```
Checkbox()
Checkbox(String str)
Checkbox(String str, boolean state)
Checkbox(String str, boolean state, CheckboxGroup grp)
Checkbox(String str, CheckboxGroup grp, boolean state)
```

Methods

```
void addItemListener(ActionListener al)
void removeItemListener(ActionListener al)

boolean getState()
void setState(boolean value)

String getLabel()
void setLabel(String str)

CheckboxGroup getCheckboxGroup()
void setCheckboxGroup(CheckboxGroup grp)
```

Methods

```
ItemEvent(ItemSelectable src, int type, Object item,
int state)

Object getItem()

ItemSelectable getItemSelectable()

int getStateChange()

void addItemListener(ItemListener i1)
Object[] getSelectedObjects()
void removeItemListener(ItemListener i1)

void itemStateChanged(ItemEvent ie)
```



Check Boxes

```
import java.applet.*;
import java.awt.*;
import java.awt.event.*;
/*
<applet code="CheckboxEvents" width=400
height=60>
</applet>
*/

public class CheckboxEvents extends Applet
implements ItemListener {
    Label label;

    public void init() {
        Checkbox cb1 = new Checkbox("Apple");
        cb1.addItemListener(this);
        add(cb1);
        Checkbox cb2 = new Checkbox("Banana");
        cb2.addItemListener(this);
        add(cb2);
        Checkbox cb3 = new Checkbox("Orange");
        cb3.addItemListener(this);
        add(cb3);
        label = new Label("      ");
        add(label);
    }
}
```

```
public void itemStateChanged(ItemEvent ie) {
    Checkbox cb =
    (Checkbox)ie.getItemSelectable();
    label.setText(cb.getLabel() + " " +
    cb.getState());
}
}
```

☐ Apple ☐ Banana ☐ Orange Applet



Check Box Groups

☐ Apple ☒ Banana ☐ Orange ☐ Apple tree

CheckboxGroup Constructor

CheckboxGroup()

Methods

Checkbox getSelectedCheckbox()

Void setSelectedCheckbox(Checkbox cb)

```
import java.applet.*;
import java.awt.*;
import java.awt.event.*;
/*
<applet code="CheckboxGroupEvents" width=400
height=60>
</applet>
*/

public class CheckboxGroupEvents extends Applet
implements ItemListener {
    Label label;
```

```
public void init() {
    CheckboxGroup cbg = new
    CheckboxGroup();
    Checkbox cb1 = new Checkbox("Apple",
    cbg, true);
    cb1.addItemListener(this);
    add(cb1);
    Checkbox cb2 = new Checkbox("Banana",
    cbg, false);
    cb2.addItemListener(this);
    add(cb2);
    Checkbox cb3 = new Checkbox("Orange",
    cbg, false);
    cb3.addItemListener(this);
    add(cb3);
    label = new Label("      ");
    add(label);
}

public void itemStateChanged(ItemEvent ie)
{
    Checkbox cb =
    (Checkbox)ie.getItemSelectable();
    label.setText(cb.getLabel() + " " +
    cb.getState());
}
}
```



Choices

Choice Constructor

Choice()

How to use a choice

1. Create a Choice object
2. Add items to the choice
3. Register the applet
4. Invoke the add() methods
5. Implement the ItemListener

```
import java.applet.*;
import java.awt.*;
import java.awt.event.*;
/*
<applet code="ChoiceEvents" width=400
height=60>
</applet>
*/

public class ChoiceEvents extends Applet
implements ItemListener {
    Label label;
```

```
public void init() {
    Choice c1 = new Choice();
    c1.addItem("Red");
    c1.addItem("Orange");
    c1.addItem("Yellow");
    c1.addItem("Green");
    c1.addItem("Blue");
    c1.addItem("Indigo");
    c1.addItem("Violet");
    c1.addItemListener(this);
    add(c1);
    Choice c2 = new Choice();
    c2.addItem("North");
    c2.addItem("South");
    c2.addItem("East");
    c2.addItem("West");
    c2.addItemListener(this);
    add(c2);
    label = new Label("      ");
    add(label);
}

public void itemStateChanged(ItemEvent ie)
{
    Choice c = (Choice)ie.getItemSelectable();
    label.setText(c.getSelectedItem());
}
}
```

TextFields and TextAreas

TextField Constructors

```
TextField()
TextField(String str)
TextField(int cols)
TextField(String str, int cols)
```

TextArea Constructor

```
TextArea()
TextArea(String str)
TextArea(int rows, int cols)
TextArea(String str, int rows, int cols)
TextArea(String str, int rows, int cols, int scrollbars)
```

Methods

```
void append(String str)
void insert(String str, int index)
void replaceRange(String str, int start, int end)
TextEvent(Object src, int type)
void textValueChanged(TextEvent te)
```

```
import java.applet.*
import java.awt.*;
import java.awt.event.*
/*
<applet code="TextFieldEvents" width=400
height=200>
</applet>
*/
```

```
public class TextFieldEvents extends Applet
implements ActionListener, TextListener {
    TextArea ta;
    TextField tf;
```

```
public void init() {
    tf = new TextField(20);
    tf.addActionListener(this);
    tf.addTextListener(this);
    ta = new TextArea(10, 20);
    add(tf);
    add(ta);
}
```

```
public void actionPerformed(ActionEvent ae) {
    ta.append("ActionEvent: " +
        ae.getActionCommand() + "\n");
    tf.setText("");
}
```

```
public void textValueChanged(TextEvent te) {
    ta.append("TextEvent: " +
        tf.getText() + "\n");
}
```



Lists

List Constructors

```
List()
List(int rows)
List(int rows, boolean multiple)
```

```
import java.applet.*;
import java.awt.*;
import java.awt.event.*;
/*
<applet code="ListEvents" width=400 height=200>
</applet>
*/
```

```
public class ListEvents extends Applet
implements ActionListener, ItemListener {
    TextArea ta;
```

```
public void init() {
    List list = new List();
    list.add("Hydrogen");
    list.add("Helium");
    list.add("Carbon");
    list.add("Oxygen");
    list.add("Potassium");
```

```
list.add("Phosphorus");
list.addActionListener(this);
list.addItemListener(this);
add(list);
ta = new TextArea(10, 20);
add(ta);
}
```

```
public void actionPerformed(ActionEvent ae) {
    ta.append("ActionEvent: " +
        ae.getActionCommand() + "\n");
}
```

```
public void itemStateChanged(ItemEvent ie) {
    List list = (List)ie.getItemSelectable();
    ta.append("ItemEvent: " +
        list.getSelectedItem() + "\n");
}
```





Scroll Bars

Scrollbar Constructors

Scrollbar()

Scrollbar(int orientation)

Scrollbar(int orientation, int value, int width, int min, int max)

AdjustmentEvent Constructor

AdjustmentEvent(Adjustable src, int id, int type, int value)

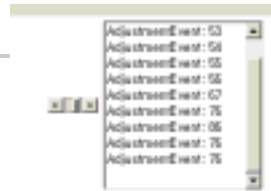
Methods

Adjustable getAdjustable()

int getAdjustmentType()

int getValue()

void adjustmentValueChanged(AdjustmentEvent ae)



```
public class ScrollbarEvents extends Applet
implements AdjustmentListener {
    TextArea ta;

    public void init() {
        Scrollbar sb =
            new Scrollbar(Scrollbar.HORIZONTAL, 50, 5, 0,
                100);
        sb.addAdjustmentListener(this);
        add(sb);
        ta = new TextArea(10, 20);
        add(ta);
    }

    public void adjustmentValueChanged(AdjustmentEvent
        ae)
    {
        Scrollbar sb = (Scrollbar)ae.getAdjustable();
        ta.append("AdjustmentEvent: " + sb.getValue() +
            "\n");
    }
}
```



Layout Managers

getLayout(), setLayout() Method

LayoutManager getLayout()

Void setLayout(LayoutManager lm)

FlowLayout Constructors

FlowLayout()

FlowLayout(int align)

FlowLayout(int align, int hgap, int vgap)

```
import java.applet.*;
import java.awt.*;
/*
<applet code="FlowLayoutApplet" width=300
height=200>
</applet>
*/

public class FlowLayoutApplet extends Applet {

    public void init() {
        setLayout(new FlowLayout(FlowLayout.LEFT, 5, 5));
        for (int i = 0; i < 20; i++) {
            add(new Button("Button" + i));
        }
    }
}
```





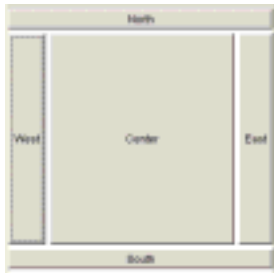
Border Layout

BorderLayout Constructors

`BorderLayout()`
`BorderLayout(int hgap, int vgap)`

add() Method

`void add(Component comp, Object obj)`



```
import java.applet.*;
import java.awt.*;
/*
<applet code="BorderLayoutApplet" width=300
height=300>
</applet>
*/
```

```
public class BorderLayoutApplet extends Applet {

    public void init() {
        setLayout(new BorderLayout(5, 5));
        Button b1 = new Button("North");
        Button b2 = new Button("South");
        Button b3 = new Button("East");
        Button b4 = new Button("West");
        Button b5 = new Button("Center");
        add(b1, "North");
        add(b2, "South");
        add(b3, "East");
        add(b4, "West");
        add(b5, "Center");
    }
}
```



GridLayout

GridLayout Constructors

`GridLayout()`
`GridLayout(int rows, int cols)`
`GridLayout(int rows, int cols, int hgap, int vgap)`

Insets Constructor

`Insets(int top, int left, int bottom, int right)`

The Insets class encapsulates information about the top, left, bottom, and right margins around the boundary of a container.

In order to specify the insets for a container, it needs to override its `getInsets()` method.



```
import java.applet.*;
import java.awt.*;
/*
<applet code="GridLayoutApplet" width=200
height=200>
</applet>
*/
```

```
public class GridLayoutApplet extends Applet {

    public void init() {
        setLayout(new GridLayout(3, 2));
        Button b1 = new Button("A");
        Button b2 = new Button("B");
        Button b3 = new Button("C");
        Button b4 = new Button("D");
        Button b5 = new Button("E");
        Button b6 = new Button("F");
        add(b1);
        add(b2);
        add(b3);
        add(b4);
        add(b5);
        add(b6);
    }

    public Insets getInsets() {
        return new Insets(10, 10, 10, 10);
    }
}
```




Panels

Panels

Panel class extends Container, and is a Component.

Layout manager is associated with a panel and determines the placement of those components.



```
public class PanelDemo extends Applet {

    public void init() {

        // Set Layout Manager
        setLayout(new BorderLayout());

        // Create panel for "North"
        Panel pn = new Panel();
        Checkbox cb1 = new Checkbox("Red", true);
        pn.add(cb1);
        Checkbox cb2 = new Checkbox("Green", false);
        pn.add(cb2);
        Checkbox cb3 = new Checkbox("Blue", false);
        pn.add(cb3);
        add(pn, "North");

        // Create panel for "Center"
        Panel pc = new Panel();
        pc.setLayout(new GridLayout(3, 2));
        for (int i = 0; i < 6; i++)
            pc.add(new Button("Button " + i));
        add(pc, "Center");

        // Create panel for "South"
        Panel ps = new Panel();
        Label label = new Label("This is the South Panel");
        ps.add(label);
        add(ps, "South");
    }
}
```



Windows and Frames

Window Constructor

Windows(Frame parent)

Methods

```
void addWindowListener(WindowListener w1)
void removeWindowListener(WindowListener w1)

void pack()

void show()

void dispose()

Window getWindow()
```

Frame Constructor

Frame()
Frame(String title)

Methods

```
MenuBar getMenuBar()
Void setMenuBar(MenuBar mb)

String getTitle()
Void setTitle(String str)

• To use Frame
1. Create a subclass of Frame.
2. Handle the window closing event of that subclass
3. Create an application or applet, invoke the show() and setSize()
```

A Frame Example

```
import java.applet.*;
import java.awt.*;
import java.awt.event.*;

class Frame2 extends Frame {

    Frame2(String title) {
        super(title);
        addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent we) {
                dispose();
            }
        });
    }
}

public class FrameApplet extends Applet
implements ActionListener {

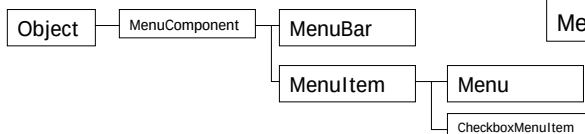
    public void init() {
        Button b = new Button("Create Frame");
        b.addActionListener(this);
        add(b);
    }

    public void actionPerformed(ActionEvent ae) {
        Frame2 f2 = new Frame2("Frame2");
        f2.setSize(200, 200);
        f2.show();
    }
}
```



Menus and Menu Bars

Menu Component Class



An Example:
MenuItemEvents.java

Methods and Constructor

void setFont(Font f)

MenuBar() → Constructor

MenuBar add(Menu m)

Menu(String str) → Constructor

MenuItem add(MenuItem mi)

Void add(String str)

MenuItem(String str) → Constructor

Methods and Constructor

void addActionListener (ActionListener a1)

void removeActionListener(ActionListener a1)

void setEnabled(boolean flag)

CheckboxMenuItem(String str) → Constructor

CheckboxMenuItem(String str, boolean flag)

void addItemListener(ItemListener i1)

void removeItemListener(ItemListener i1)

boolean getState()

void setState(boolean flag)



Dialogs and File Dialogs

Dialog Constructors & Method

Dialog(Frame parent)
Dialog(Frame parent, boolean flag)
Dialog(Frame parent, String title)
Dialog(Frame parent, String title, boolean flag)

void show()

How to create a dialog

1. Define a subclass of Dialog
2. Inside the Constructor : create and arrange user interface elements, invoke pack()
3. Call dispose when finished with dialog.

FileDialog Constructors & Methods

FileDialog(Frame parent)
FileDialog(Frame parent, String str)
FileDialog(Frame parent, String str, int rw)

String getFile()
void setFile(String str)

How to use dialog

1. Instantiate the Dialog subclass
2. Call setVisible() to make the dialog visible.
3. Call setSize() to set the dimensions of the dialog